TEC | Tecnológico de Costa Rica

Escuela de Administración de Tecnologías de Información

TI-3600 Bases de datos

Profesora:

Jacqueline Tatiana Solis Cespedes

Proyecto #2

Licitación para el Ecosistema Digital de Cines IMAX

Elaborado por:

Jiménez Loría Fabricio | 2025101456

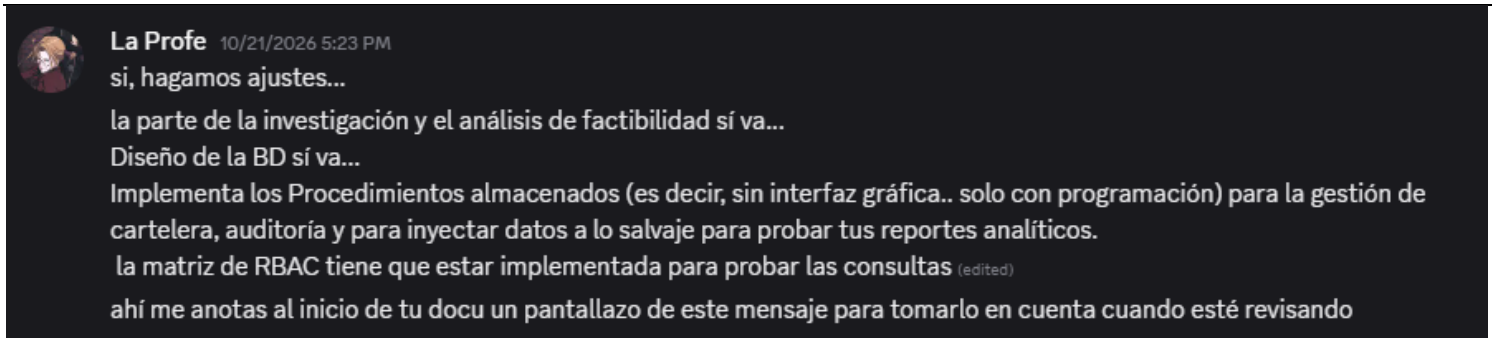
Cartago

II Semestre 2026

Tabla de contenido

1. Dictamen de rechazo y propuesta de factibilidad técnica 4
1.1. Contexto y motivos para rechazar la infraestructura anterior 4
1.2. Propuesta de una arquitectura que pueda crecer 4
1.3. Factibilidad técnica 5
1.4. Factibilidad operativa 6
1.5. Riesgos principales y medidas para reducirlos 6
1.6. Factibilidad económica y comparación entre proveedores 7
1.7. Proyección financiera a tres años 8
1.8. Costo mensual por millón de transacciones 8
2. Análisis de requisitos y dimensionamiento de datos 9
2.1. Requisitos funcionales 9
2.2. Requisitos no funcionales 9
2.3. Datos y carga de trabajo esperados 10
2.4. Proyección del historial de boletos 10
2.5. Estimación de almacenamiento de registros y auditoría 11
2.6. Qué significan estos cálculos para la propuesta 11
3. Documentación del diseño de la base de datos 11
3.1. Enfoque general y diagramas 11
3.2. Organización de las tablas del sistema 12
3.3. Relaciones principales 12
3.4. Claves, reglas de integridad y normalización 12
3.5. Controles implementados y su alcance 13
3.6. Vistas analíticas y rendimiento 13
3.7. Posible organización futura de los datos históricos 14
4. Operación, seguridad y mantenimiento propuestos 14
4.1. Respaldos y recuperación de la base de datos 14
4.2. Metas de recuperación y disponibilidad 15
4.3. Manejo seguro de pagos y datos personales 15
4.4. Responsabilidades del personal 16
4.5. Registro y conservación de auditorías 16
5. Matriz RBAC y pruebas realizadas 16
5.1. Matriz de roles solicitada por el profesor 16
5.2. Implementación y límites del control de acceso 17

5.3. Pruebas realizadas y evidencias..... 18
6. Conclusiones 18
Fuentes consultadas..... 19



1. Dictamen de rechazo y propuesta de factibilidad técnica

1.1. Contexto y motivos para rechazar la infraestructura anterior

IMAX necesita un sistema que pueda manejar ventas de boletos y operaciones de cines en distintos países. El caso plantea un escenario bastante exigente: durante un estreno podrían conectarse 8 millones de personas al mismo tiempo y generarse hasta 50.000 operaciones por segundo durante diez minutos. También se habla de 500 películas, 1.500 salas y unos 15 millones de boletos al mes.

La propuesta del proveedor anterior se basa en Access o SQLite instalados en una computadora antigua, internet doméstico de 50 Mbps y una hoja de cálculo compartida para llevar el control de asientos. Incluso propone enviar datos completos de tarjetas por mensajería y reiniciar el equipo cuando algo falle. Para un sistema de esta escala, esa forma de trabajar presenta varios problemas.

Dependencia de un solo equipo. Si la computadora o la conexión se cae, las ventas podrían detenerse por completo. Además, no se contempla un sistema alternativo para continuar trabajando mientras se resuelve el problema.

Riesgo de vender el mismo asiento dos veces. Una hoja de cálculo compartida no controla las reservas simultáneas de la misma manera que una base de datos con transacciones y validaciones. Esto podría provocar que dos clientes terminen comprando el mismo asiento.

Manejo inseguro de los pagos. Compartir números de tarjeta y códigos CVV por mensajes expone información sensible. Además, PCI DSS prohíbe almacenar el CVV después de autorizar una compra. En nuestro prototipo los pagos son simulados; en una solución real habría que utilizar una pasarela de pago segura.

Falta de un plan de recuperación. Reiniciar una computadora no asegura que se conserven los datos ni que el servicio vuelva rápidamente. Hacen falta respaldos, responsables claros y un procedimiento para recuperarse de una falla.

Por todo esto, considero que la propuesta anterior debe rechazarse. No es que Access o SQLite sean malas herramientas, sino que la infraestructura planteada no responde a las necesidades de seguridad, continuidad y volumen que tiene IMAX en este caso.

1.2. Propuesta de una arquitectura que pueda crecer

Mi propuesta es utilizar PostgreSQL como base de datos principal y apoyarse en servicios en la nube que permitan ampliar recursos cuando haya mucha demanda. En lugar de depender de una sola computadora, el sistema se dividiría en componentes para atender las distintas tareas y recuperarse mejor ante posibles fallas.

Parte del sistema	Propuesta	Para qué sirve
Entrada de usuarios	Balanceador de carga, red de distribución (CDN) y WAF	Distribuye las solicitudes y ayuda a filtrar ataques o tráfico sospechoso.
Procesos del negocio	Servicios de cartelera, reservas, compras y administración	Permite ampliar los servicios de acuerdo con la demanda.
Base de datos	PostgreSQL administrado con alta disponibilidad	Mantiene los datos de funciones, asientos y ventas con reglas y transacciones.
Registros históricos	Almacenamiento separado para auditoría y logs	Permite conservar auditorías históricas sin sobrecargar las consultas del día.
Continuidad	Copias automáticas, monitoreo y recuperación	Facilita la recuperación si ocurre una falla.

Para evitar ventas duplicadas, el sistema debe comprobar la disponibilidad del asiento antes de confirmar una reserva y aplicar controles en la base de datos. También debe registrar los cambios importantes, de forma que se pueda revisar qué sucedió y quién realizó cada operación.

Esta arquitectura representa cómo podría funcionar una plataforma real a futuro. En este proyecto se trabajó con PostgreSQL de manera local; no se está afirmando que ya exista una instalación internacional funcionando en AWS o Azure.

1.3. Factibilidad técnica

Desde el punto de vista técnico, la propuesta es viable. Tanto AWS como Azure cuentan con servicios de bases de datos, almacenamiento, seguridad y recuperación que pueden combinarse para crear una plataforma más estable. Además, PostgreSQL permite mantener relacionadas las películas, funciones, salas, reservas, compras y boletos.

Eso sí, contratar servicios en la nube no significa que automáticamente se puedan atender 8 millones de usuarios. Para comprobarlo habría que simular un estreno con mucha demanda, medir el comportamiento de cada componente y verificar que no se dupliquen las ventas de asientos.

El enunciado también establece metas de recuperación y disponibilidad: un RTO menor de 5 minutos, un RPO de 5 segundos o menos y una disponibilidad del 99,99 %. En este trabajo se toman como objetivos de diseño, no como resultados que ya se hayan conseguido.

1.4. Factibilidad operativa

En cuanto a la operación diaria, IMAX necesitaría personas encargadas de revisar el sistema, actualizarlo, controlar accesos, hacer respaldos y resolver incidentes. Un proveedor de nube puede simplificar parte del mantenimiento, pero no reemplaza esas responsabilidades.

También es importante separar las tareas del personal. Por ejemplo, un taquillero no debería cambiar precios generales y un contador no necesita modificar compras. Los administradores, por su parte, deben dejar un registro cuando hacen cambios delicados. Para manejar estas diferencias se utiliza el control de acceso basado en roles, conocido como RBAC.

Para el curso, esta propuesta resulta posible porque PostgreSQL y pgAdmin permiten desarrollar y probar las reglas del sistema sin pagar infraestructura empresarial. Lo que construimos es un prototipo académico, no una plataforma mundial ya puesta en producción.

1.5. Riesgos principales y medidas para reducirlos

La siguiente tabla reúne los riesgos que consideré más importantes y las medidas que podrían ayudar a reducirlos. Las probabilidades son estimaciones para analizar el caso; no corresponden a estadísticas reales de incidentes de IMAX.

Riesgo	Prob.	Impacto	Medida propuesta	Recuperación objetivo
R-01 · Caída de la BD	Media	Crítico	Alta disponibilidad en dos zonas, réplica, respaldos y ensayo de recuperación.	Meta RTO < 5 min; RPO ≤ 5 s.
R-02 · Venta duplicada	Media	Crítico	Transacciones, restricciones y prueba simultánea sobre un mismo asiento.	Evitar duplicados; corregir incidentes sin pérdida de datos.
R-03 · Saturación de demanda	Alta	Crítico	Repartir solicitudes, caché para cartelera y ampliación de recursos.	Meta de volver a operar: < 5 min.
R-04 · Caída del pago	Media	Alto	Confirmación de la pasarela, reintentos controlados y liberación de asientos.	No confirmar ventas sin pago; restablecer el proceso.
R-05 · Robo de información	Media	Crítico	Cifrado, WAF, acceso mínimo y pasarela externa; no guardar CVV.	Contener y notificar; RTO/RPO no describen por sí solos una filtración.

Riesgo	Prob.	Impacto	Medida propuesta	Recuperación objetivo
R-06 · Alteración de precios	Media	Alto	Matriz RBAC, registro de responsable y revisión de cambios sensibles.	Restaurar valores válidos y conservar evidencia.
R-07 · Pérdida de auditoría	Media	Crítico	Logs separados, almacenamiento que impida cambios y conservación de 5 años.	Meta de protección: RPO ≤ 5 s; restauración probada.
R-08 · Falla regional	Baja	Crítico	Copia en otra región y plan formal para reanudar el servicio.	Objetivo RTO/RPO sujeto a pruebas; no garantizado por réplicas asíncronas.

Las metas de RTO y RPO solo podrían confirmarse después de configurar los servicios y hacer pruebas de recuperación. Por eso, ninguno de estos valores se presenta como garantizado por el simple hecho de elegir un proveedor.

1.6. Factibilidad económica y comparación entre proveedores

Para comparar los costos se utilizaron los montos de ejemplo incluidos en el enunciado. Se analizaron AWS y Microsoft Azure, ya que ambos ofrecen PostgreSQL administrado y servicios para almacenamiento histórico, procesamiento y protección de solicitudes.

Servicio necesario	AWS	Microsoft Azure
Base de datos relacional	Amazon Aurora PostgreSQL	Azure Database for PostgreSQL Flexible Server
Servicios de procesamiento	Amazon EKS o servicio equivalente	Azure Kubernetes Service o equivalente
Auditoría durante cinco años	Amazon S3 y clases Glacier	Azure Blob Storage y nivel Archive
Distribución y seguridad	CloudFront, balanceador y AWS WAF	Azure Front Door y WAF

El costo real de una solución en la nube cambia según la región, la capacidad contratada, el almacenamiento y la cantidad de tráfico. Por eso, los montos de las siguientes tablas sirven como referencia académica y no deben interpretarse como cotizaciones oficiales de AWS o Azure.

Componente mensual	AWS	Azure
Base de datos	US$ 12.500	US$ 11.800
Procesamiento de servicios	US$ 6.200	US$ 5.900
Almacenamiento histórico y logs	US$ 1.800	US$ 1.750
Seguridad y red / WAF	US$ 3.500	US$ 3.400
Total mensual (OPEX)	US$ 24.000	US$ 22.850

Con los valores del ejemplo, Azure resulta US$ 1.150 más barato al mes que AWS. De todas formas, antes de tomar una decisión real habría que comparar configuraciones equivalentes: misma capacidad, región, respaldos y volumen de tráfico. Para hacerlo se pueden utilizar las calculadoras oficiales: https://calculator.aws/ y https://azure.microsoft.com/en-us/pricing/calculator/.

1.7. Proyección financiera a tres años

El CAPEX es el dinero que se invierte al inicio para preparar e implementar el sistema, mientras que el OPEX corresponde a los gastos para mantenerlo funcionando. En esta comparación se utilizó un CAPEX de US$ 15.000, tomado del ejemplo del profesor, y se supuso que el OPEX crecería un 10 % cada año.

Concepto	AWS	Azure
CAPEX inicial (una vez)	US$ 15.000	US$ 15.000
OPEX mensual inicial	US$ 24.000	US$ 22.850
OPEX del año 1	US$ 288.000	US$ 274.200
OPEX del año 2 (+10 %)	US$ 316.800	US$ 301.620
OPEX del año 3 (+10 %)	US$ 348.480	US$ 331.782
TOTAL a tres años, incluido CAPEX	US$ 968.280	US$ 922.602

Para realizar la proyección, primero se multiplicó el gasto mensual por 12. Luego se aumentó el resultado un 10 % para el segundo año y otro 10 % para el tercero. Al sumar esos tres años más el CAPEX inicial, Azure presenta un costo aproximado de US$ 45.678 menos que AWS.

La comparación permite tener una idea del gasto, pero todavía no es un presupuesto definitivo. No incluye impuestos, comisiones de las pasarelas de pago ni cambios de precios. Tampoco se puede calcular un retorno de inversión confiable sin saber cuánto dinero ganaría o ahorraría IMAX con el nuevo sistema.

1.8. Costo mensual por millón de transacciones

El proyecto pide calcular cuánto costaría atender un millón de transacciones al mes. Aunque el caso indica que se venden 15 millones de boletos mensuales, no especifica cuántas operaciones se realizan por cada venta. Para hacer la comparación supuse cuatro operaciones principales por boleto: reserva, pago, confirmación de compra y emisión del boleto.

Operaciones mensuales estimadas = 15.000.000 boletos × 4 operaciones = 60.000.000 operaciones. El costo por millón se obtiene dividiendo el gasto mensual entre 60.

Indicador	AWS	Azure
OPEX mensual de referencia	US$ 24.000	US$ 22.850
Operaciones mensuales supuestas	60.000.000	60.000.000
Costo por 1 millón de operaciones	US$ 400,00	US$ 380,83

Este cálculo es una aproximación: no incluye consultas de cartelera, reintentos, errores ni otras solicitudes. Tampoco representa una tarifa oficial de los proveedores. Para obtener un dato más preciso habría que medir las operaciones reales y utilizar una configuración concreta en las calculadoras de costos.

En resumen, PostgreSQL junto con servicios en la nube es una alternativa razonable para este caso. Azure aparece como la opción de menor costo dentro del ejercicio, pero una decisión real dependería de precios actualizados y de pruebas de capacidad y recuperación.

2. Análisis de requisitos y dimensionamiento de datos

2.1. Requisitos funcionales

Los requisitos funcionales son las tareas que el sistema debe permitir. Para IMAX se tomaron las operaciones principales relacionadas con cartelera, asientos, compras, boletos, reportes y seguridad.

Código	Requisito	Descripción
RF-01	Gestionar cartelera	Registrar, modificar y consultar películas y funciones, mostrando solo las funciones activas que todavía no comienzan.
RF-02	Administrar salas y asientos	Definir las salas, su capacidad y los asientos disponibles para cada función.
RF-03	Reservar sin duplicaciones	Permitir seleccionar asientos y evitar que dos compras queden confirmadas para el mismo asiento.
RF-04	Registrar ventas y pagos	Guardar reservas, compras y pagos; cuando un pago falle, permitir liberar los asientos correspondientes.
RF-05	Emitir y validar boletos	Generar un identificador único por boleto y evitar que sea validado dos veces.
RF-06	Aplicar precios	Manejar tarifas y descuentos de acuerdo con las reglas del cine.
RF-07	Consultar reportes	Obtener ocupación, recaudación, funciones con mayor demanda y métodos de pago.
RF-08	Controlar los permisos	Permitir o rechazar operaciones según el rol de cada usuario.
RF-09	Registrar auditoría	Guardar el historial de cambios importantes, reembolsos, bloqueos y accesos a las salas.

La profesora aclaró que no hace falta desarrollar una interfaz gráfica. Por eso, las funciones se implementaron y demostraron mediante procedimientos almacenados y consultas SQL, manteniendo las mismas reglas del negocio.

2.2. Requisitos no funcionales

Los requisitos no funcionales indican cómo debería comportarse el sistema en temas como seguridad, velocidad, disponibilidad y mantenimiento.

Código	Requisito	Criterio esperado
RNF-01	Capacidad	Diseñarse tomando como referencia hasta 8 millones de usuarios concurrentes y un pico de 50.000 operaciones por segundo.
RNF-02	Disponibilidad	Proponer una disponibilidad de 99,99 % y opciones de recuperación ante fallas.
RNF-03	Recuperación	Tomar como metas RTO menor a 5 minutos y RPO de 5 segundos o menos.
RNF-04	Integridad	Impedir ventas duplicadas y mantener consistentes reservas, pagos y boletos.

Código	Requisito	Criterio esperado
RNF-05	Privacidad y seguridad	Restringir el acceso según el rol y no guardar CVV ni información bancaria innecesaria.
RNF-06	Auditoría	Conservar registros históricos durante al menos cinco años.
RNF-07	Facilidad de mantenimiento	Mantener reglas y procedimientos SQL organizados y documentados para facilitar cambios y revisiones.

2.3. Datos y carga de trabajo esperados

La siguiente tabla muestra las cifras establecidas en el enunciado. Son datos del escenario que se utiliza para diseñar la propuesta, no resultados alcanzados durante las pruebas del curso.

Dato solicitado	Valor del escenario	Para qué se utiliza
Usuarios simultáneos	8.000.000	Planear cómo repartir las conexiones y consultas.
Pico de operaciones	50.000 por segundo	Dimensionar la capacidad durante los estrenos.
Duración del pico	10 minutos	Calcular el volumen concentrado de solicitudes.
Títulos activos	500	Dimensionar el catálogo de películas.
Salas a nivel mundial	1.500	Dimensionar las funciones y el inventario de asientos.
Boletos mensuales	15.000.000	Calcular el crecimiento de ventas y sus registros.
Retención de auditoría	5 años	Estimar almacenamiento histórico y respaldos.

Si el sistema llegara a trabajar a 50.000 operaciones por segundo durante diez minutos, tendría que atender 30.000.000 de operaciones (50.000 × 60 × 10). Eso no significa que se vendan 30 millones de boletos, porque también hay consultas y otras solicitudes. Del mismo modo, 8 millones de usuarios conectados no implican 8 millones de conexiones directas a PostgreSQL; se necesita una capa intermedia que distribuya el trabajo.

2.4. Proyección del historial de boletos

Si IMAX mantuviera 15 millones de boletos vendidos al mes durante cinco años, se acumularían unos 900.000.000 de boletos (15.000.000 × 12 × 5). Es una proyección simple que no incluye crecimiento ni descuenta boletos anulados. Aun así, ayuda a entender por qué el almacenamiento histórico debe planificarse desde el inicio.

Periodo	Boletos estimados
1 mes	15.000.000
1 año	180.000.000
3 años	540.000.000
5 años	900.000.000

2.5. Estimación de almacenamiento de registros y auditoría

El enunciado pide conservar durante cinco años los registros de pagos, bloqueos de asientos, accesos mediante QR y cambios administrativos. Como no se especifica cuánto ocupa cada registro, para esta estimación se utilizaron tamaños aproximados y se muestran los cálculos correspondientes.

Los supuestos utilizados fueron los siguientes: 450 bytes por registro de pago, tres movimientos de asiento de 220 bytes por boleto, 1,2 intentos de ingreso de 200 bytes por boleto, 100.000 cambios administrativos mensuales de 400 bytes y 250 bytes para el historial de cada boleto. Son valores estimados para dimensionar el escenario, no mediciones realizadas sobre las tablas del prototipo.

Tipo de información	Cálculo mensual	Estimación GB/mes
Pagos y transacciones	15.000.000 × 450 bytes	6,75
Bloqueos y estados de asiento	15.000.000 × 3 × 220 bytes	9,90
Escaneos y accesos con QR	15.000.000 × 1,2 × 200 bytes	3,60
Cambios administrativos	100.000 × 400 bytes	0,04
Historial de boletos	15.000.000 × 250 bytes	3,75
Subtotal sin índices ni margen	Suma de categorías	24,04
Margen técnico adicional (35 %)	24,04 GB × 0,35	8,41
Total estimado por mes	24,04 GB × 1,35	32,45

Los cálculos están expresados en GB decimales, donde 1 GB equivale a 1.000.000.000 de bytes. Se agregó un margen del 35 % para contemplar índices, organización y crecimiento interno. Ese porcentaje es una decisión del ejercicio, no una cifra fija de PostgreSQL.

Con los supuestos anteriores, el cálculo da 32,454 GB al mes. En cinco años serían 1.947,24 GB, es decir, cerca de 1,95 TB. Antes de aplicar el margen, los registros de pagos, asientos, QR y administración suman unos 20,29 GB mensuales y el historial de boletos aporta otros 3,75 GB.

Si el almacenamiento creciera un 10 % cada año, la estimación acumulada rondaría los 2,38 TB en cinco años. A eso habría que sumar respaldos, réplicas y otros archivos del sistema. Los 45 TB mencionados en el ejemplo económico del profesor son una referencia ilustrativa y no salen directamente de estos cálculos.

2.6. Qué significan estos cálculos para la propuesta

Estos números muestran que IMAX necesitaría dos cosas: una base de datos que responda rápido a las ventas actuales y un espacio aparte para conservar el historial. La idea es que los registros antiguos puedan pasar a un almacenamiento más económico sin perderse ni quedar expuestos a cambios no autorizados.

Para convertir esta estimación en una cotización real, primero habría que medir el tamaño de los registros con más datos, considerar los intentos fallidos y reintentos de pago, revisar el tráfico de cartelera y definir la política de respaldos. Por ahora, los números se presentan como una aproximación razonada para el caso.

3. Documentación del diseño de la base de datos

3.1. Enfoque general y diagramas

El diseño se trabajó en tres niveles. El modelo conceptual muestra la información principal del negocio y sus relaciones: películas, cines, salas, funciones, asientos, usuarios, compras y boletos. El modelo lógico organiza

esas relaciones en tablas. Finalmente, el modelo físico muestra cómo quedaron implementadas en PostgreSQL, con sus tipos de datos, claves y validaciones.

Los diagramas del proyecto se encuentran como archivos separados en el repositorio. Para describir la implementación también se tomaron como referencia 01_estructura_base.sql y 02_IMAX_estructura_final.sql, de manera que el informe corresponda con el esquema desarrollado.

3.2. Organización de las tablas del sistema

Grupo	Tablas creadas	Propósito
Cartelera y salas	pelicula, cine, sala, asiento, funcion	Guarda la cartelera, las salas y su distribución de asientos.
Venta y reservas	usuario, reserva, reserva_asiento, compra, pago, boleto	Relaciona a cada usuario con sus reservas, compras, pagos y boletos.
Precios y devoluciones	tarifa, promocion, reembolso	Permite definir precios, promociones y devoluciones.
Entrada al cine	validacion_qr	Lleva el historial de validaciones de boletos en la entrada.
Roles y vigilancia	rol, permiso, usuario_rol, rol_permiso, auditoria	Guarda los roles, permisos y cambios auditados.

Al diseño original de 20 tablas se le añadió cuenta_postgres_usuario. Esta tabla permite relacionar las cuentas de PostgreSQL con los usuarios del sistema IMAX, lo cual resulta útil tanto para controlar el acceso a los procedimientos como para registrar quién hizo una operación. En total, el modelo final tiene 21 tablas.

3.3. Relaciones principales

Las relaciones siguen la forma en que funciona un cine. Cada cine puede tener varias salas y cada sala contiene asientos. Una película puede proyectarse en distintas funciones, y cada función tiene una sala asignada. Las reservas pertenecen a un usuario y a una función, mientras que reserva_asiento indica los asientos elegidos. A partir de una reserva se registra la compra, sus pagos y los boletos. Las validaciones QR se vinculan a cada boleto y la auditoría permite revisar las operaciones realizadas.

En la parte de seguridad, usuario_rol relaciona a cada usuario con sus roles y rol_permiso indica qué puede hacer cada rol. Estas tablas intermedias permiten que un usuario tenga varios roles sin duplicar la información.

3.4. Claves, reglas de integridad y normalización

Las tablas cuentan con claves primarias para identificar cada registro y claves foráneas para mantener las relaciones. Por ejemplo, no se puede registrar una función de una película que no existe ni vincular un boleto a una compra inexistente. Algunas tablas utilizan claves formadas por dos columnas, como la relación entre reservas y asientos.

También se definieron reglas para evitar datos repetidos, como correos de usuarios duplicados, códigos QR iguales o dos asientos con la misma fila y número dentro de una sala. Las restricciones CHECK ayudan a validar duraciones, montos y fechas, entre otros datos.

La información se separó para no guardar lo mismo en varias tablas. Por ejemplo, el título de una película se almacena en pelicula y la función guarda su identificador, sin repetir todos sus datos. Esto sigue criterios de

normalización, aunque para asegurar formalmente que todo el esquema cumple tercera forma normal (3FN) habría que revisar las dependencias funcionales una por una.

3.5. Controles implementados y su alcance

En el esquema inicial se incluyeron funciones y triggers que protegen las reservas y las relaciones entre salas y asientos. Por ejemplo, ciertos cambios se bloquean cuando una reserva ya tiene asientos asignados. También se agregó una restricción que impide registrar más de una validación QR aprobada para el mismo boleto.

Procedimientos almacenados desarrollados

Para las operaciones principales se implementaron procedimientos en PL/pgSQL. Estos permiten utilizar el sistema desde PostgreSQL y probar sus reglas sin desarrollar una interfaz gráfica.

Los procedimientos que se implementaron son:

- registrar_pelicula: registra películas desde una cuenta con permisos de administración.
- programar_funcion: permite programar una función y comprueba que no se cruce con otra en la misma sala.
- reservar_asiento: crea reservas temporales y usa bloqueos transaccionales para controlar intentos simultáneos sobre un asiento.
- iniciar_compra: inicia una compra pendiente a partir de una reserva válida.
- procesar_pago_simulado: simula la aprobación o el rechazo del pago; si se aprueba, confirma la reserva y emite los boletos.
- validar_boleto_qr: revisa el boleto, su horario y su estado para impedir que un mismo código sea aprobado dos veces.
- cargar_ventas_prueba: genera reservas, compras, pagos y boletos ficticios para probar los reportes.

También probamos qué ocurría cuando dos conexiones distintas intentaban reservar el mismo asiento. En ese escenario, los bloqueos de PostgreSQL evitaron que ambas confirmaran reservas incompatibles.

3.6. Vistas analíticas y rendimiento

Vista existente	Información que devuelve	Resultado verificado
vista_ocupacion_funciones	Capacidad de sala, boletos vendidos y porcentaje de ocupación por función.	Se comprobaron ocupaciones del 12,5 % y del 30 %.
vista_recaudacion_funciones	Ingresos brutos, reembolsos procesados e ingreso neto por función.	Se comprobaron ₡30.000 y ₡72.000 por función.

Vista existente	Información que devuelve	Resultado verificado
vista_top_5_funciones	Cinco funciones con mayor cantidad de boletos contabilizados.	Las cinco funciones registraron 12 boletos cada una.
vista_metodos_pago	Cantidades de pagos aprobados/rechazados e importes por método.	377 pagos aprobados, 35 rechazados y ₡2.262.000.

Resultados de los reportes

Se probaron las cuatro vistas analíticas con los datos que se cargaron en la base:

- vista_ocupacion_funciones: mostró funciones con 5 boletos de 40 asientos (12,5 % de ocupación) y otras con 12 boletos de 40 asientos (30 %).
- vista_recaudacion_funciones: calculó ₡30.000 para funciones con cinco boletos y ₡72.000 para funciones con doce, a un precio de ₡6.000 por boleto.
- vista_top_5_funciones: presentó cinco funciones con 12 boletos vendidos cada una.
- vista_metodos_pago: mostró 377 pagos aprobados, 35 rechazados y ₡2.262.000 recaudados.

Además, se crearon índices en columnas que se utilizan para relacionar reservas, pagos, reembolsos y asientos. Se revisaron los tiempos con EXPLAIN ANALYZE, pero con el tamaño actual de la base no se observó una mejora clara. Por eso, estos resultados demuestran que las consultas funcionan; no prueban que el sistema soporte por sí solo la carga de una plataforma internacional.

EXPLAIN ANALYZE permite revisar cómo ejecuta PostgreSQL una consulta y cuánto tarda. Lo usamos para comparar el comportamiento antes y después de crear índices, aunque para sacar conclusiones de rendimiento más firmes se necesitarían más datos y condiciones de prueba comparables.

Que aparezca un Seq Scan no significa necesariamente que haya un problema: con pocas filas, recorrer la tabla puede ser más rápido que usar un índice. Por eso, la utilidad de los índices debe evaluarse con mediciones.

3.7. Posible organización futura de los datos históricos

En una plataforma mundial convendría separar la información de uso diario de los registros históricos de pagos, bloqueos de asientos, validaciones QR y cambios administrativos. El caso pide conservar estos datos durante cinco años. Una opción sería organizarlos por categoría y por fecha, aunque esas tablas históricas especializadas y su particionamiento todavía son parte de la propuesta y no de la implementación local.

Esta separación permitiría consultar las ventas actuales sin tener que recorrer años de información. Los cálculos del capítulo 2 dan una idea inicial del espacio requerido, pero habría que medir los tamaños reales cuando se trabaje con mayores volúmenes.

4. Operación, seguridad y mantenimiento propuestos

4.1. Respaldos y recuperación de la base de datos

Los respaldos permiten recuperar información si ocurre un error o una falla. Para una operación real de IMAX se propone programar copias completas y conservar los registros de cambios de PostgreSQL para intentar recuperar la información hasta un momento cercano al incidente. También sería importante guardar copias en otra ubicación y hacer pruebas periódicas de restauración.

Medida propuesta	Frecuencia orientativa	Responsable o verificación
Copia de respaldo completa	Cada día, fuera del horario de mayor demanda.	Administración de base de datos; revisar que finalizó sin errores.
Registro de cambios (WAL) y recuperación puntual	Archivo continuo de los cambios, según el servicio contratado.	Monitorear retrasos y puntos de recuperación disponibles.
Copia protegida fuera de la región principal	Según la política de recuperación y el proveedor.	Comprobar acceso y posibilidad de restauración.
Prueba de restauración	Al menos una vez al mes; después de cambios importantes.	Anotar tiempo de recuperación y datos recuperados.
Revisión de capacidad y costos	Mensualmente.	Ajustar espacio, índices y presupuesto.

Las frecuencias de la tabla son recomendaciones, no una configuración que ya esté activa. Por ejemplo, hacer un respaldo diario no basta para alcanzar un RPO de cinco segundos: se necesitaría proteger los cambios casi en tiempo real y comprobarlo con pruebas.

4.2. Metas de recuperación y disponibilidad

Medida	Objetivo del enunciado	Significado práctico
RTO	Menos de 5 minutos.	Volver a ofrecer el servicio después de una interrupción.
RPO	5 segundos o menos.	Perder como máximo el equivalente a cinco segundos de datos.
SLA	99,99 % de disponibilidad.	Aproximadamente 4,32 minutos de caída permitidos en un mes de 30 días.

Tener una réplica en otra zona puede ayudar a reducir las interrupciones y la pérdida de datos, especialmente cuando las escrituras se confirman en ambos lados. Sin embargo, si falla una región completa, una copia remota podría no estar totalmente actualizada. Por eso, los objetivos de RTO y RPO necesitan una configuración concreta y pruebas antes de considerarse cumplidos.

La documentación de Azure explica que la alta disponibilidad entre zonas puede recuperar el servicio rápidamente y evitar la pérdida de datos confirmados en determinadas configuraciones. AWS también ofrece recuperación entre regiones, aunque esta puede tardar varios minutos y depender del retraso de replicación. Son alternativas para valorar, no garantías que ya tengamos en el proyecto.

4.3. Manejo seguro de pagos y datos personales

Para los pagos reales, lo adecuado sería utilizar una pasarela especializada. IMAX no debería guardar códigos CVV ni enviar datos bancarios por mensajería. También se necesitarían cuentas individuales, permisos adecuados, conexiones cifradas y respaldos protegidos para cuidar los datos personales y financieros.

Cuando alguien modifica precios, cancela una operación o fuerza un reembolso, debería quedar registrado quién lo hizo y cuándo. Para conservar esos registros durante cinco años se pueden considerar opciones como S3 Object Lock o el almacenamiento inmutable de Azure. Eso sí, estas medidas requieren configuración y no vienen funcionando automáticamente por contratar el servicio.

4.4. Responsabilidades del personal

Perfil	Responsabilidad principal	Límite esperado
Administrador o gerente	Configurar películas, salas, tarifas y autorizaciones.	Sus cambios delicados deben registrarse.
Taquillero	Realizar ventas presenciales y consultar su corte de caja.	No cambia la configuración general ni la auditoría.
Operador de puerta	Validar el ingreso utilizando los códigos QR.	No realiza ventas ni modifica precios.
Contador	Consultar y conciliar ingresos.	No modifica las funciones ni la venta de asientos.
Cliente	Consultar funciones y comprar sus propias entradas.	No accede a administración ni a reportes internos.

4.5. Registro y conservación de auditorías

El caso distingue cuatro tipos de información que deben conservarse: pagos, bloqueos de asientos, ingresos con QR y cambios administrativos. En un sistema a gran escala sería conveniente separarlos y protegerlos durante cinco años para poder consultarlos sin que se alteren.

En el prototipo ya existen las tablas auditoria y validacion_qr. Lo que todavía no se implementó fue una separación completa de bitácoras ni una política automática para conservarlas durante cinco años.

Para registrar los cambios se creó la función auditar_cambios_imax y se configuraron seis triggers en las tablas pelicula, funcion, compra, pago, boleto y validacion_qr. Estos están preparados para las operaciones INSERT, UPDATE y DELETE, y relacionan la acción con el usuario identificado mediante su cuenta PostgreSQL.

Durante las pruebas se encontraron 1.422 registros de auditoría, principalmente asociados con inserciones. Esto demuestra que el registro automático funciona en los casos probados; sin embargo, conservar los registros de forma inmutable durante cinco años sigue siendo una parte pendiente de la propuesta empresarial.

5. Matriz RBAC y pruebas realizadas

5.1. Matriz de roles solicitada por el profesor

La matriz siguiente resume los cinco perfiles pedidos en el enunciado. Los permisos se implementaron parcialmente en PostgreSQL y se probaron algunas operaciones permitidas y denegadas. Las funciones que faltan se mantienen como parte del alcance futuro.

Función	Cliente	Taquillero	Puerta	Contador	Admin.
Cartelera	Ver	Ver	Ver	Ver	Gestionar
Asientos y compra	Comprar	Vender	—	—	Configurar
Emisión / QR	Ver propios	Emitir	Validar	—	Resolver casos
Reportes	—	Caja propia	—	Finanzas	Todos
Salas y funciones	—	—	—	—	Gestionar
Auditoría y roles	—	—	—	—	Gestionar

El símbolo (—) indica que ese perfil no debería acceder a la función. Aunque el administrador tiene permisos más amplios, también debe respetar controles del sistema, especialmente cuando existen reservas o ventas activas.

5.2. Implementación y límites del control de acceso

Cómo se comprobó el RBAC

Se crearon cinco roles de PostgreSQL: imax_administrador, imax_cliente, imax_taquillero, imax_contador e imax_operador_sala. Además, se configuraron cuentas individuales para comprobar qué puede hacer cada tipo de usuario.

Los procedimientos tienen permisos de ejecución definidos. En las pruebas, el cliente pudo reservar asientos, pero no registrar películas ni validar códigos QR. Por otro lado, la cuenta de administración sí pudo registrar películas, mientras que el operador de sala pudo validar boletos.

También se hicieron doce comprobaciones de permisos sobre tablas protegidas. En todos esos casos, las cuentas evaluadas tenían bloqueadas las operaciones directas de insertar, modificar y eliminar datos.

Estas pruebas muestran que el control por roles funciona en las operaciones revisadas. Aun así, algunas tareas de taquilla y administración no se desarrollaron completamente, por lo que el RBAC no se presenta como una solución final para todos los perfiles.

5.3. Pruebas realizadas y evidencias

Prueba	Resultado obtenido
Gestión de cartelera	Se registraron películas y se programaron funciones desde procedimientos almacenados.
Reservas y concurrencia	Dos conexiones distintas intentaron reservar el mismo asiento y se comprobó el bloqueo.
Compras y pagos	Se registraron 415 reservas y 412 compras; 377 pagos fueron aprobados y 35 rechazados.
Emisión de boletos	Durante las pruebas se emitieron 377 boletos.
Validación QR	Un boleto válido pudo usarse y el segundo intento fue rechazado.
Seguridad RBAC	Se probaron permisos permitidos y denegados, además de doce controles de acceso a tablas.
Auditoría	Se comprobaron 1.422 eventos y seis triggers de auditoría.
Carga de datos	Se trabajó con 50 películas, 10 salas, 364 asientos y 182 funciones.
Reportes analíticos	Funcionaron los reportes de ocupación, recaudación, métodos de pago y top 5.
Rendimiento	Se revisaron índices y planes con EXPLAIN ANALYZE; la mejora de velocidad no fue concluyente.

Las compras pagadas sumaron ₡2.262.000. Al comparar los reportes con los registros de pagos y boletos, los resultados coincidieron con los datos de prueba.

La prueba de concurrencia se hizo con dos conexiones independientes. No se probaron entre 20 y 50 clientes simultáneos ni los volúmenes internacionales del enunciado, por lo que sería incorrecto afirmar que ya se alcanzó esa capacidad.

Para respaldar lo realizado, se subieron nueve capturas al repositorio de GitHub. Allí se pueden ver los resultados de ventas, los permisos RBAC, la auditoría, los triggers y los cuatro reportes.

6. Conclusiones

La infraestructura del proveedor anterior no responde a las necesidades de seguridad, continuidad y cantidad de usuarios que plantea el caso de IMAX. Por eso, se propone trabajar con PostgreSQL y servicios en la nube que permitan aumentar recursos, controlar accesos y recuperar datos ante fallas. Tanto AWS como Azure podrían servir, aunque Azure resulta más económico únicamente con los precios de ejemplo del ejercicio.

En la parte práctica se logró implementar y probar varias operaciones importantes desde PostgreSQL: reservas de asientos, pagos simulados, emisión y validación de boletos QR, controles RBAC y auditoría automática. Todo esto se realizó mediante procedimientos almacenados, sin necesidad de crear una interfaz gráfica.

Las pruebas dejaron 377 pagos aprobados, 35 rechazados y una recaudación de ₡2.262.000. También se verificaron cuatro reportes analíticos y se comprobó el control de reservas simultáneas utilizando dos conexiones independientes.

En general, el proyecto permitió construir una base funcional para el sistema de cines IMAX y comprobar sus reglas principales a escala académica. Para llevarlo a una operación internacional harían falta pruebas de carga más grandes, un entorno de alta disponibilidad y ensayos de recuperación ante fallas. Por ahora, esos puntos quedan planteados como pasos siguientes y no como resultados ya alcanzados.

Fuentes consultadas

Amazon Web Services. Calculadora de precios de AWS. https://calculator.aws/

Microsoft Azure. Calculadora de precios. https://azure.microsoft.com/en-us/pricing/calculator/

Amazon Web Services. Precios de Amazon Aurora. https://aws.amazon.com/rds/aurora/pricing/

Microsoft Azure. Precios de Azure Database for PostgreSQL. https://azure.microsoft.com/en-us/pricing/details/postgresql/flexible-server/

Amazon Web Services. Amazon S3 Pricing. https://aws.amazon.com/s3/pricing/

Microsoft Azure. Azure Blob Storage Pricing. https://azure.microsoft.com/en-us/pricing/details/storage/blobs/

Amazon Web Services. AWS WAF Pricing. https://aws.amazon.com/waf/pricing/

Microsoft Azure. Azure Front Door Pricing. https://azure.microsoft.com/en-us/pricing/details/frontdoor/

PostgreSQL. Documentación oficial: Backup and Restore. https://www.postgresql.org/docs/current/backup.html

PostgreSQL. Documentación oficial: Continuous Archiving and Point-in-Time Recovery. https://www.postgresql.org/docs/current/continuous-archiving.html

PostgreSQL. Documentación oficial: Using EXPLAIN. https://www.postgresql.org/docs/current/using-explain.html

Microsoft Learn. Alta disponibilidad de Azure Database for PostgreSQL. https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-high-availability

Microsoft Learn. Continuidad del negocio y recuperación. https://learn.microsoft.com/en-us/azure/postgresql/backup-restore/concepts-business-continuity

Amazon Web Services. Aurora Global Database: recuperación. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database-disaster-recovery.html

Amazon Web Services. Protección con S3 Object Lock. https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

Microsoft Learn. Almacenamiento inmutable de Azure Blob. https://learn.microsoft.com/es-es/azure/storage/blobs/immutable-storage-overview

PCI Security Standards Council. Información sobre códigos de verificación de tarjetas. https://www.pcisecuritystandards.org/faqs/1319/